

Project 1: Threads - Design Document

CSE 421/521 - Operating Systems | Fall 2026

Qiang Wu <qwu26@buffalo.edu>

Mohammad Sahir <msahir@buffalo.edu>

Siraat Mustafa <siraatmu@buffalo.edu>

Alarm Clock is implemented. Priority Scheduling and Advanced Scheduler are planned designs.

Technical references: the supplied course assignment and the Pintos Project 1 manual and 4.4BSD Scheduler appendix.

Pintos Project 1 manual

Pintos 4.4BSD Scheduler appendix

Alarm Clock

A1. Data structures

Copy here the declaration of each new or changed `struct' or `struct' member, global or static variable, `typedef', or enumeration. Identify the purpose of each in 25 words or less.

Added in devices/timer.c:

```
struct sleep_waiter
{
    uint64_t wake_tick;
    struct semaphore wake;
    struct list_elem elem;
};

static struct list sleepers;
```

sleep_waiter	A sleep request stored on the calling thread's stack.
--------------	---

wake_tick	The tick when the thread can become ready.
-----------	--

wake	A semaphore that blocks the thread until the timer wakes it.
------	--

elem	Connects the request to the sleep list.
------	---

sleepers	The pending requests, sorted by wake_tick.
----------	--

```
struct thread is unchanged.
```

A2. Sleep and wakeup

Briefly describe what happens in a call to timer_sleep(), including the effects of the timer interrupt handler.

timer_sleep() returns immediately if the duration is zero or negative. Otherwise, it initializes a local semaphore to zero. With interrupts disabled, it calculates the wakeup tick and inserts the request into the sorted sleep list. It then restores interrupts and calls sema_down() to block.

The timer interrupt checks the front of the list. It removes each request whose wakeup tick has arrived and calls sema_up() on its semaphore. It stops at the first request that is not due. This wakes all threads due on the same tick.

A3. Interrupt cost

What steps are taken to minimize the amount of time spent in the timer interrupt handler?

The list is sorted when a thread goes to sleep, so the interrupt handler only visits requests that are due, plus the first one that is not. It does not scan the rest of the list. Inserting a request costs $O(n)$; waking k requests costs $O(k + 1)$.

A4. Concurrent sleep calls

How are race conditions avoided when multiple threads call `timer_sleep()` simultaneously?

Each thread has its own request and semaphore. Interrupts are disabled only while reading ticks and inserting into the shared list. On this single-CPU system, another thread cannot run during those operations. The caller restores interrupts before waiting on its semaphore.

A5. Timer races

How are race conditions avoided when a timer interrupt occurs during a call to `timer_sleep()`?

The request is initialized before it is added to the list. Interrupts are disabled during insertion, so the handler cannot see an unfinished request.

The timer may signal after interrupts are restored but before `sema_down()`. In that case, `sema_up()` leaves the semaphore value at one, and `sema_down()` consumes it without blocking. If `sema_down()` runs first, the thread blocks until the timer signals it.

The handler removes the request before signaling. Its stack storage stays valid while `timer_sleep()` is blocked, and it is no longer in the list when the function returns.

A6. Design choice

Why did you choose this design? In what ways is it superior to another design you considered?

A semaphore lets each thread block and handles a signal that arrives before the thread starts waiting. Keeping the request on the stack avoids allocating memory or adding fields to struct thread.

An unsorted list would make insertion faster, but the timer would have to scan every request on every tick. Sorting at insertion keeps that work out of the interrupt handler. The cost is $O(n)$ insertion with interrupts disabled, which gets more expensive as the list grows.

Priority Scheduling

B1. Planned data structures

Copy here the declaration of each new or changed `struct` or `struct` member, global or static variable, `typedef`, or enumeration. Identify the purpose of each in 25 words or less.

Planned additions to struct thread in threads/thread.h:

```
int base_priority;
struct lock *waiting_lock;
struct list held_locks;
struct list_elem lock_wait_elem;
```

base_priority	The priority requested by the thread before donation.
---------------	---

waiting_lock	The lock this thread is trying to acquire, or NULL.
--------------	---

held_locks	Locks currently owned by this thread.
------------	---------------------------------------

lock_wait_elem	Links an unfinished acquisition into the lock's contenders.
----------------	---

The existing priority member will hold the effective scheduling priority. Add a forward declaration of struct lock in thread.h.

Planned additions to struct lock in threads/synch.h:

```
struct list contenders;
struct list_elem held_elem;
```

contenders	Threads whose acquisition attempts have not yet succeeded.
------------	--

held_elem	Links this lock into its holder's held_locks list.
-----------	--

Planned addition to semaphore_elem in threads/synch.c:

```
struct thread *thread;
```

thread	Identifies a condition waiter before its private semaphore blocks.
--------	--

The existing ready_list and semaphore waiters lists will be reused. These declarations belong to the planned implementation.

B2. Donation tracking

Explain the data structure used to track priority donation. Use ASCII art to diagram a nested donation. (Alternately, submit a .png file.)

Each thread will keep its base priority separately from its effective priority. held_locks identifies the locks from which it can receive donations. Each lock's contenders list holds threads whose acquisition attempts have not yet succeeded, linked through lock_wait_elem. waiting_lock connects a waiting thread to the next lock in the chain.

A contender will remain registered after it is unblocked and until it actually obtains the lock. The semaphore wait list alone cannot represent that interval. A new holder will therefore see donations from every remaining acquisition attempt.

For H, M, and L with base priorities 50, 30, and 10:

H (base 50)	M (base 30)	L (base 10)
waits for A ----->	holds A	
	waits for B ----->	holds B
effective 50	effective 50	effective 50

The effective priority is the maximum of the base priority and the current effective priorities of contenders for all held locks. Donation will follow waiting_lock and holder links through at most eight holders. A null holder or repeated thread stops the traversal.

B3. Priority selection and preemption

How do you ensure that the highest priority thread waiting for a lock, semaphore, or condition variable wakes up first?

We will append ready threads and semaphore waiters in arrival order. When selecting a thread, we will scan the list using its current effective priority and keep the first maximum. A strictly greater priority replaces the candidate; an equal priority does not. Equal-priority threads therefore keep FIFO order, and a yielding thread joins the ready list at the tail.

sema_up() will select the highest-priority blocked waiter, remove it from the wait list, unblock it, and increment the semaphore value before any ordinary context switch. The lock's semaphore uses the same selection. After the surrounding operation finishes its metadata updates, a higher priority ready thread will cause a yield. An interrupt handler will request intr_yield_on_return() instead of yielding inside the handler. thread_create() will also check preemption after making a new thread ready. Idle will yield to any ready thread, even one with priority zero. Ordinary thread_unblock() will keep its atomic contract; its callers will check preemption once their metadata updates are complete.

cond_signal() will scan condition waiters through their saved thread pointers and use the same current-priority rule. This also works when a signal arrives before the waiter's sema_down(), while its private semaphore still has an empty wait list. sema_up() signals that waiter's private semaphore; a blocked waiter becomes ready, while an early signal is retained until its next sema_down(). The waiter must reacquire the monitor lock before cond_wait() returns.

B4. Lock acquisition

Describe the sequence of events when a call to lock_acquire() causes a priority donation. How is nested donation handled?

If the lock is unavailable, lock_acquire() will set the caller's waiting_lock and link its acquisition attempt into the lock's contenders. It will recompute the holder's effective priority from the holder's base priority and all contenders for its held locks.

If the holder is waiting for another lock, the recomputation will continue to that lock's holder. This propagates a high-priority donation through a nested chain, up to the eight-holder limit. The caller will then wait on the lock semaphore. These waiting steps use sema_down() and thread_block(), not a loop that repeatedly yields.

After the caller successfully takes the semaphore, it will remove its contender entry if it registered one and clear waiting_lock. It will assign the new holder and add the lock to held_locks, including on an uncontended acquisition. It will recompute its effective priority from any remaining contenders. A successful lock_try_acquire() will use the same ownership bookkeeping; a failed try will not register or donate. The registration, ownership, and priority changes will use the existing synchronization primitives' short atomic sections, with blocking delegated to the semaphore.

B5. Lock release

Describe the sequence of events when `lock_release()` is called on a lock that a higher-priority thread is waiting for.

`lock_release()` will remove the lock from the current thread's `held_locks` and clear the lock's holder. It will recompute the releasing thread's effective priority using its base priority and contenders for its remaining locks. This removes donations for this lock while preserving others.

It will then release the semaphore, selecting its highest-priority blocked waiter. Contender records remain until their threads actually acquire the lock. A new holder will inherit donations from remaining contenders, including a contender already made ready but not yet successful.

All release bookkeeping and the semaphore increment will finish before a normal-context preemption check. If a ready thread has a higher priority, the releasing thread will yield immediately. Releasing a lock makes its waiter ready; dispatch still chooses the highest-priority ready thread, which can be another thread.

B6. Priority-update race

Describe a potential race in `thread_set_priority()` and explain how your implementation avoids it. Can you use a lock to avoid this race?

A race could occur if `thread_set_priority()` reads the old donation state, then is preempted before storing the effective priority. Another thread could add a higher donation while the setter is suspended. Writing the old calculated value after resuming would discard that donation.

We will protect the base-priority change, donation scan, effective-priority write, and ready-priority comparison in one short atomic update. The setter will change `base_priority` and retain any larger active donation. Any yield will happen after the metadata update is complete. `thread_get_priority()` will return the effective value.

An extra sleeping lock is unsuitable for this scheduler update. Acquiring it could block and introduce a donation dependency into the mechanism that maintains donations, and interrupt handlers cannot acquire it. Short interrupt exclusion protects this internal state on Pintos's single CPU. Semaphores and locks still provide the actual blocking and waiting.

B7. Design choice

Why did you choose this design? In what ways is it superior to another design you considered?

Grouping donation sources by held lock makes releasing one lock clear: its donations disappear while the other locks still contribute. Keeping an acquisition in contenders until success also covers the gap between wakeup and ownership, without allocating a separate donation object.

Selecting from current priorities avoids keeping every ready and waiting list sorted after each donation. A sorted-list alternative could reduce selection cost, but would need to update a thread's position whenever its effective priority changes, including while blocked on another semaphore.

The cost of our plan is linear selection and scans of held locks and their contenders. It also adds list links to threads and locks. This is a simple starting point for the project; priority buckets or cached lock maxima could reduce those scans later, at the cost of more update paths.

Advanced Scheduler

C1. Planned data structures

Copy here the declaration of each new or changed `struct` or `struct` member, global or static variable, `typedef`, or enumeration. Identify the purpose of each in 25 words or less.

Planned declarations in threads/fixed-point.h:

```
typedef int32_t fixed_t;
#define FP_SHIFT 14
#define FP_SCALE (1 << FP_SHIFT)
```

fixed_t	Stores a signed 17.14 fixed-point number.
FP_SHIFT	The number of fractional bits.
FP_SCALE	The scale factor for integer and fixed-point conversions.

Planned additions to struct thread in threads/thread.h:

```
int nice;
fixed_t recent_cpu;
```

nice	The thread's scheduling preference, inherited from its creator.
recent_cpu	The thread's recent CPU usage, including its fractional part.

Planned global in threads/thread.c:

```
static fixed_t load_avg;
```

load_avg	The system-wide moving average of runnable non-idle threads.
----------	--

The existing priority member, ready_list, all_list, and thread_mlfqs will be reused. A single ready list will select the first current maximum priority, preserving FIFO order among equal priorities. Fixed-point helpers will use int64_t intermediate products and dividends. These declarations belong to the planned implementation.

C2. Scheduling example

Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a recent_cpu value of 0. Fill in the table below showing the scheduling decision and the priority and recent_cpu values for each thread after each given number of timer ticks:

Tick	recent_cpu			Priority			Next thread
	A	B	C	A	B	C	
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A
8	8	0	0	61	61	59	B
12	8	4	0	61	60	59	A
16	12	4	0	60	60	59	B
20	12	8	0	60	59	59	A
24	16	8	0	59	59	59	C
28	16	8	4	59	59	58	B
32	16	12	4	59	58	58	A
36	20	12	4	58	58	58	C

C3. Table conventions

Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

The table assumes that A, B, and C are initially ready in that order, that they remain runnable, and that each time slice lasts four ticks. A row shows recent_cpu and priorities after accounting for the listed ticks and recalculating priorities, then identifies the thread selected for the next slice. At tick zero, it shows the initial selection.

An equal-priority choice depends on ready-list order. Our plan appends an expired-slice thread behind its existing peers and selects the first current maximum. For example, B precedes A at tick 8, and C precedes both at tick 24. A can keep running if it still has strictly higher priority, as at tick 4.

This is the rule the implementation will use. The table is a calculation of the proposed policy, not output from an implemented MLFQS scheduler. No one-second load_avg or recent_cpu decay occurs within these 36 ticks with TIMER_FREQ equal to 100.

C4. Update order and performance

How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

Each timer tick will charge one recent_cpu unit to the running non-idle thread. Alarm wakeups will follow. At a one-second boundary, load_avg will be updated first, followed by recent_cpu for every non-idle thread, including blocked threads. At each global four-tick boundary, all non-idle priorities will be recalculated. A tick that matches both intervals will use the new load_avg and recent_cpu in its priority calculation.

The calculations will use:

```
priority = PRI_MAX - recent_cpu / 4 - 2 * nice
load_avg = (59/60) * load_avg + (1/60) * ready_threads
recent_cpu = (2*load_avg)/(2*load_avg+1) * recent_cpu + nice
```

The ready-thread count will be taken after alarm wakeups, so it includes threads whose deadlines have arrived at that boundary, plus the running non-idle thread. All these operations will finish before an interrupt return can schedule an ordinary thread. The current thread_tick() and timer handler will be separated into accounting and periodic updates to produce this order.

The per-thread periodic scans cost $O(n)$ for n threads. Ready selection costs $O(r)$ for r ready threads, and the alarm adds $O(k+1)$ for k due requests. Keeping all-thread scans at the specified boundaries avoids doing them every tick, but those boundary interrupts are still longer. Thread creation and thread_set_nice() will calculate the affected thread's priority outside interrupt context; their shared-state changes will use short atomic sections.

C5. Design tradeoffs

Briefly critique your design, pointing out advantages and disadvantages in your design choices. If you were to have extra time to work on this part of the project, how might you choose to refine or improve your design?

Reusing the ready list gives both scheduling policies the same selection rule. We will read priorities at dispatch rather than rely on a list that was sorted before the last periodic update. This also lets blocked threads receive updated recent_cpu and priority without moving between queues.

The tradeoff is $O(r)$ dispatch and $O(n)$ periodic scans. Sixty-four FIFO ready queues could make dispatch cheaper, but changed priorities would require moving ready threads while preserving the chosen tie order. If the thread count made scans expensive, that would be the first optimization to consider. The simple list is our initial design.

Initialization will set the first thread's nice and recent_cpu to zero. Later ordinary threads will inherit both values before they are published in all_list. Idle will be recognized before it becomes ready and excluded from these calculations. In MLFQS mode, the explicit creation priority and thread_set_priority() will be ignored, and donation will be bypassed. thread_set_nice() will recalculate immediately and check preemption.

After implementation, the nine MLFQS tests will check load, fairness, niceness, recent CPU accounting, and blocked-thread behavior. Additional review will cover idle startup, inherited state, negative arithmetic, and ticks matching both update boundaries.

C6. Fixed-point arithmetic

The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

We will use a signed 17.14 fixed-point type with scale $F = 1 \ll 14$. Named helpers in `threads/fixed-point.h` will handle conversion and arithmetic, keeping scale corrections out of the scheduler formulas. A header of small inline functions is sufficient; no separate source file or build-rule change is needed.

An integer n becomes $n * F$. Adding an integer uses $x + n * F$. Two fixed-point values multiply as $((\text{int64_t } x * y) / F$ and divide as $((\text{int64_t } x * F) / y$. The wide intermediate avoids overflow during scaling. Multiplication and division by F will be used instead of signed bit shifts, whose behavior is awkward for negative operands.

We will keep stored fractions intact, including a negative `recent_cpu`. The priority formula will be evaluated in fixed point, then the complete result converted toward zero and clamped to 0..63. For a valid positive priority, this conversion rounds down. The `recent_cpu/4` term will not be converted early. The coefficient for the one-second `recent_cpu` update will be calculated before multiplying by `recent_cpu`.

The recent CPU and load getters will multiply by 100 using a wide intermediate, then round to nearest. For scaled x , that conversion uses $(x + F/2)/F$ when nonnegative and $(x - F/2)/F$ when negative, with integer division toward zero. These distinct helpers will make truncation and rounding explicit at their call sites.

Optional survey questions

In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

Do you have any suggestions for the TAs to more effectively assist students, either for future quarters or the remaining projects?

Any other comments?